Estructuras de datos

Análisis de algoritmos

Jonnathan Ramos

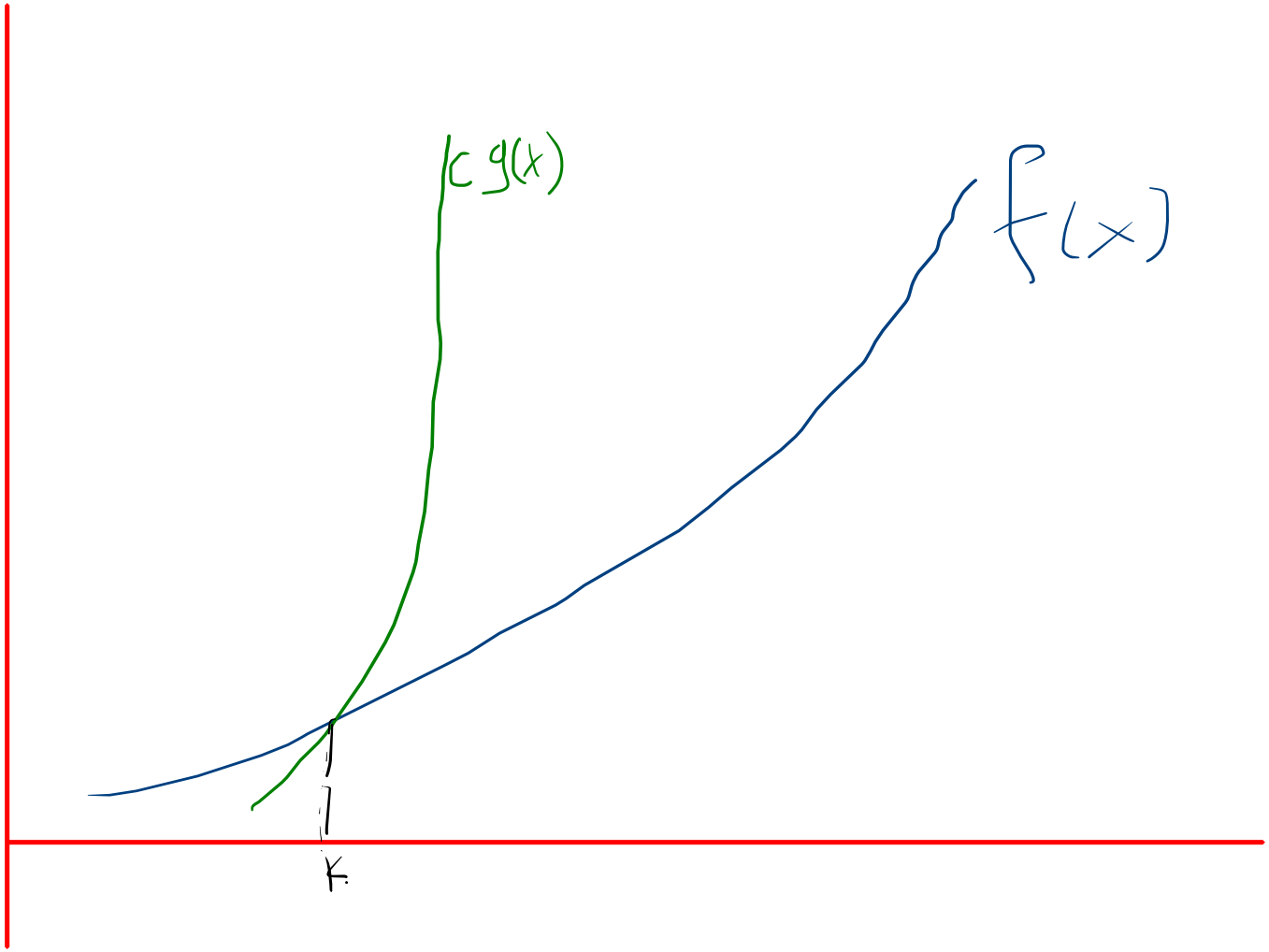
Universidad de Industrial de Santander - UIS
Facultad de Ingenierías Físico-Mecánicas
Escuela de Ingeniería de Sistemas e Informática - EISI
Ingeniería en Ciencia de datos

Definición

Sean $f(x)$ y $g(x)$ funciones de los enteros o reales en los reales. Se dice que $f(x)$ es $O(g(x))$ si existen las constantes C y k tal que:

$$|f(x)| \leq C|g(x)|$$

cuando $x > k$.


cg(x)
f(x)
k

$f(x) = 3x^2 + 2x + 3$ $\quad C = ?$ $\quad k = ?$

$f(x) = O(x^2)$ $\quad f(x) \leq C\, g(x)$

$k = 4$

$3x^2 \rightarrow 3(4)^2 = 3(4)^2 \quad 3x^2$

$+ 2x \rightarrow 2(4) = (4)^2 \quad + x^2$

$+ 3 \rightarrow 3 = (4)^2 \quad + x^2$

$f(x) \quad 5x^2$

(C, k) testigos $\quad S \; \exists (C, k) \rightarrow \exists$ infinitos testigos.

$f(x) = O(x^2)$ $\qquad$ $x^2 = O(x^3)$

$f(x) = O(x^3)$

Definición

Sean $f(x)$ y $g(x)$ funciones de los enteros o reales en los reales. Se dice que $f(x)$ es $\Omega(g(n))$ si existen las constantes C y k tal que:

$$|f(x)| \geq C|g(x)|$$

cuando $x > k$.

$f(x) = 3x^2 + 2x + 3$

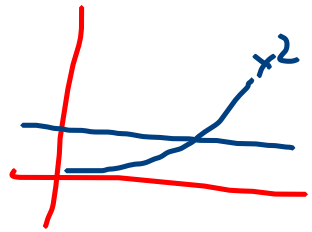
$f(x) = \Omega(x^2)$

$k = 2$

$3x^2 \geq x^2$ $\quad 2x \geq 0$ $\quad 3 \geq 0$

$f(x) = \Omega(x^2)$

$k = 1$ $\quad c = 1$



Definición

Sean $f(x)$ y $g(x)$ dos funciones del conjunto de los enteros o reales en el conjunto de los reales. Se dice que $f(x)$ es $\Theta(g(x))$ si $f(x)$ es $O(g(x))$ y $f(x)$ es $\Omega(g(x))$.

Theorem

Sean $f(n), g(n), A(n), B(n)$ funciones positivas definidas en los enteros positivos.

- **Propiedad Transitiva.** *Si $f(n) = \Theta(A(n))$ y $A(n) = \Theta(B(n))$, entonces $f(n) = \Theta(B(n))$. Pasa del mismo modo con O y Ω.*
- **Propiedades de multiplicación y cancelación.** *Si $f(n) = \Theta(A(n))$, entonces $f(n)g(n) = \Theta(A(n)g(n))$ y viceversa. Pasa del mismo modo con O y Ω.*
- **Combinación lineal.** *Sean α y β constantes positivas. Si $f(n) = \Theta(A(n))$ y $g(n) = \Theta(A(n))$, entonces $(\alpha f(n) + \beta g(n)) = \Theta(A(n))$. Pasa del mismo modo con O y Ω.*

Theorem

Sean $f(n), g(n), A(n), B(n)$ funciones positivas definidas definidas en los enteros positivos.

- **Adición y Multiplicación.** *Si $f(n) = \Theta(A(n))$ y $g(n) = \Theta(B(n))$, entonces $f(n)g(n) = \Theta(A(n)B(n))$ y $f(n) + g(n) = \Theta(A(n) + B(n))$. Más generalmente, para cualquier número fijo de funciones $f_1(n), ..., f_p(n)$ y $A_1(n), ..., A_p(n)$, si $f_k(n) = \Theta(A_k(n))$ para cada par f_k y A_k, entonces el producto $f_1(n)...f_p(n) = \Theta(A_1(n)...A_p(n))$ y $\sum_{k=a(n)}^{b(n)} f(k) = \Theta(\sum_{k=a(n)}^{b(n)} A(k))$. Pasa del mismo modo con O y Ω.*

Theorem

Sean $f(n), g(n), A(n), B(n)$ funciones positivas definidas en los enteros positivos.

- *Si $f(k) = \Theta(A(k))$, entonces $\sum_{k=1}^{n} f(k) = \Theta(\sum_{k=1}^{n} A(k))$. De un modo más general, si $a(n)$ y $b(n)$ son funciones cuyos valores son valores enteros positivos, entonces $\sum_{k=a(n)}^{b(n)} f(k) = \Theta(\sum_{k=a(n)}^{b(n)} A(k))$. Pasa del mismo modo con O y Ω.*
- **División por Θ.** *Si $f(n) = \Theta(A(n))$ y $g(n) = \Theta(B(n))$, entonces $\frac{f(n)}{g(n)} = \Theta(\frac{A(n)}{B(n)}$.*

Theorem

Sean $f(n), g(n), A(n), B(n)$ funciones positivas definidas en los enteros positivos.

- **Uso de desigualdades.**
  - *Si $f(n) \leq g(n)$ para todos los n suficientemente grandes y $g(n) = O(A(n))$, entonces $f(n) = O(A(n))$.*
  - *Si $f(n) \geq g(n)$ para todos los n suficientemente grandes y $g(n) = \Omega(A(n))$, entonces $f(n) = \Omega(A(n))$.*
  - *Si $g(n) \leq f(n) \leq h(n)$ para todos los n suficientemente grandes y $g(n) = \Theta(A(n))$ y $h(n) = \Theta(A(n))$, entonces $f(n) = \Theta(A(n))$.*

Definición Estimar → el recurso

El análisis de algoritmos es la ciencia de determinar la cantidad de recursos computacionales (usualmente tiempo y espacio) requeridos por un computador para ejecutar un algoritmo. El análisis de algoritmos permite, en el sentido práctico:

1. *Escoger el algoritmo más conveniente para la tarea.*
2. *Conocer la manera en la que escala el algoritmo para resolver la tarea.*



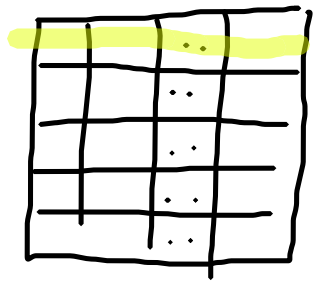
Para determinar el tiempo demandado por un algoritmo se inicia definiendo un costo para las operaciones del algoritmo y se cuenta el número de operaciones necesarias para resolver el problema para una entrada de tamaño n (una instancia del problema).

Algorithm 1: Multiplicación de matrices.

Input: Arreglo bidimensional A de $n \times t$ números.
Arreglo bidimensional B de $t \times m$ números.
Output: Arreglo bidimensional C de $n \times m$ números.

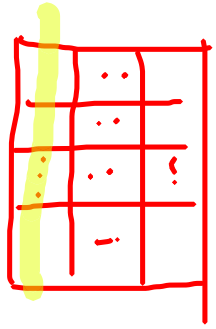
```
1 for i = 1 to n do
2     for j = 1 to m do
3         suma = 0
4         for k = 1 to t do
5             suma = suma + (A[i][k]*B[k][j])
6         C[i][j]=suma
```

```
for i = 1 to n do
    for j = 1 to m do
        suma = 0
        for k = 1 to t do
            suma = suma + (A[i][k]*B[k][j])
        C[i][j]=suma
```



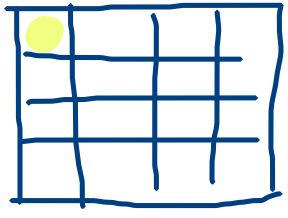
n x t

A



t x m

B



C n x m

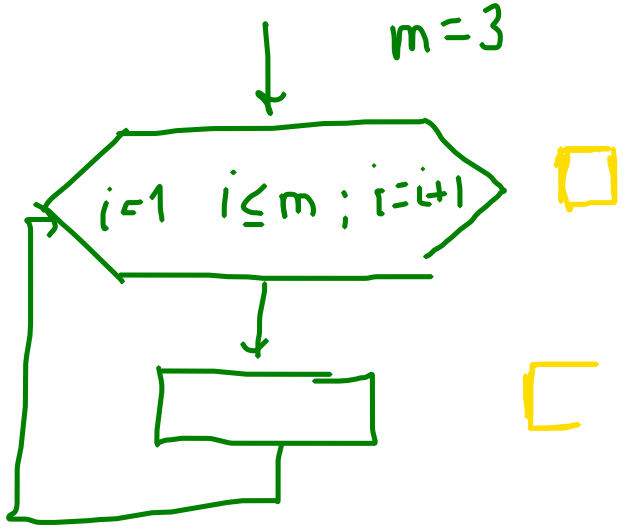
ESTIMAR TIEMPO

1ra asignación + comparación
2da → incremento + comparación

```
for i = 1 to n do                                   C1 → (n+1)
    for j = 1 to m do                               C2 → Σ_{i=1}^{n} (m+1)
        suma = 0                                    C3 → Σ_{i=1}^{n} (m)
        for k = 1 to t do                           C4 → Σ_{i=1}^{n} Σ_{j=1}^{m} (t+1)
            suma = suma + (A[i][k]*B[k][j])         C5 → Σ_{i=1}^{n} Σ_{j=1}^{m} t
        C[i][j]=suma                                C6 → Σ_{i=1}^{n} (m)
```

i = 1	i = 2	i = 3	...	i = n
(m+1)	(m+1)	(m+1)		(m+1)

i = 1				
j = 1	j = 2	j = 3	... j = m	j = m+1
t+1	t+1	t+1	t+1	



$$T(n) = (n+1)c_1 + \sum_{i=1}^{n}(m+1)c_2 + \sum_{i=1}^{n}(m)c_3 + \sum_{i=1}^{n}\sum_{j=1}^{m}(t+1)c_4 + \sum_{i=1}^{n}\sum_{j=1}^{m}t\,c_5 + \sum_{i=1}^{n}(m)c_6$$

$$T(n) = (n+1)c_1 + n(m+1)c_2 + nmc_3 + nm(t+1)c_4 + nmtc_5 + nmc_6$$

$$T(n) = nc_1 + c_1 + nmc_2 + nc_2 + nmc_3 + nmtc_4 + nmc_4 + nmtc_5 + nmc_6$$

$$T(n) = n(c_1 + c_2) + nm(c_2 + c_3 + c_4 + c_6) + nmt(c_4 + c_5) + c_1$$

$$= na + nmb + nmtc + c_1$$

con $a = c_1 + c_2$

$b = c_2 + c_3 + c_4 + c_6$

$c = c_4 + c_5$

$T(n) = O(nmt)$ Peor caso

$T(n) = \Omega(nmt)$ Mejor caso

$T(n) = \Theta(nmt)$

Si las matrices son $n \times n$

$T(n) = \Theta(n^3)$

Algorithm 2: Filtro de medias.

Input: A arreglo de $n \times t$ números

```
1 for i = 1 to n − 1 do
2     for j = 1 to t − 1 do
3         suma = 0
4         for k = i − 1 to i + 1 do
5             for m = j − 1 to j + 1 do
6                 suma = suma + A[k][m]
7         A[i][j]=suma/9
```

```
1 for i = 1 to n − 1 do
2     for j = 1 to t − 1 do
3         suma = 0
4         for k = i − 1 to i + 1 do
5             for m = j − 1 to j + 1 do
6                 suma = suma + A[k][m]
7         A[i][j]=suma/9
```

$C_1 \rightarrow n$

$C_2 \rightarrow \sum_{i=1}^{n-1} t$

$C_3 \rightarrow \sum_{i=1}^{n-1} (t-1)$

$C_4 \rightarrow \sum_{i=1}^{n-1} \sum_{j=1}^{t-1} 4$

$C_5 \rightarrow \sum_{i=1}^{n-1} \sum_{j=1}^{t-1} 3(4)$

C_6

C_7

1

$i-1 \quad i \quad i+1 \quad i+2$

$O(n \cdot t) \quad \Omega(n \cdot t)$

Si el arreglo es $n \times n$

$O(n^2) \quad \Omega(n^2)$

$\Theta(n^2)$

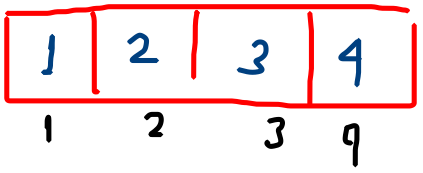
Algorithm 3: Ordenamiento por Inserción, *Insertion-Sort*

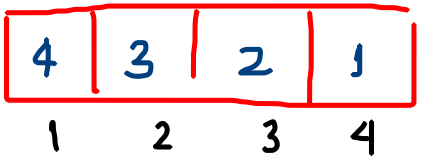
Input: Arreglo unidimensional A de n números.

Output: Arreglo de entrada ordenado.

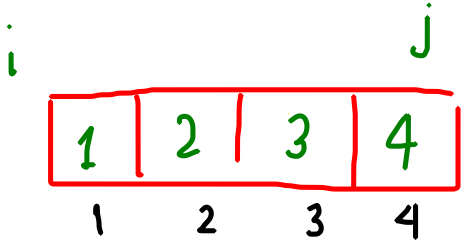
```
1 for (j = 2; j ≤ n; j++) do
2     key = A[j];
3     i = j - 1;
4     while i > 0 and A[i] > key do
5         A[i+1]=A[i];
6         i = i -1;
7     A[i+1] = key;
```

```
1 for (j = 2; j ≤ n; j++) do
2     key = A[j];
3     i = j - 1;
4     while i > 0 and A[i] > key do
5         A[i+1]=A[i];
6         i = i -1;
7     A[i+1] = key;
```





$O(n^2)$



Key = 3

$C_4 \rightarrow$ L

$C_5 \rightarrow$ |

$C_6 \rightarrow$ |

Key = 2

$C_4 \rightarrow$ ⊏

$C_5 \rightarrow$ L

$C_6 \rightarrow$ L

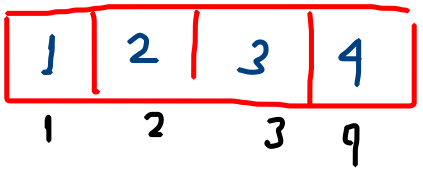
Key = 1

$C_4 \rightarrow$ □

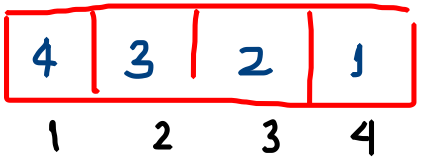
$C_5 \rightarrow$ ⊏

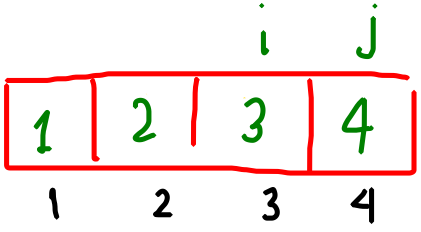
$C_6 \rightarrow$ ⊏

```
1 for (j = 2; j ≤ n; j++) do
2 |   key = A[j];
3 |   i = j - 1;
4 |   while i > 0 and A[i] > key do
5 |   |   A[i+1]=A[i];
6 |   |   i = i -1;
7 |   A[i+1] = key;
```



O(n)





Key = 2

$C_4 \rightarrow 1$

$C_5 \rightarrow$

$C_6 \rightarrow$

Key = 3

$C_4 \rightarrow 1$

$C_5 \rightarrow$

$C_6 \rightarrow$

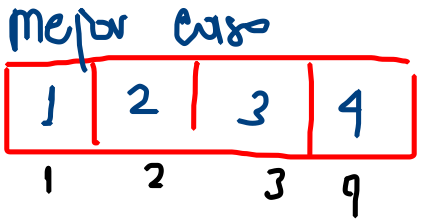
Key = 4

$C_4 \rightarrow 1$

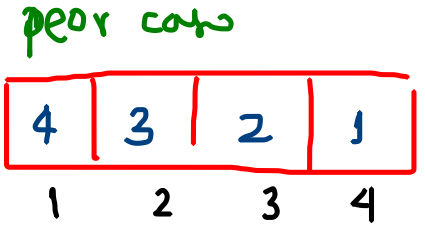
$C_5 \rightarrow$

$C_6 \rightarrow$

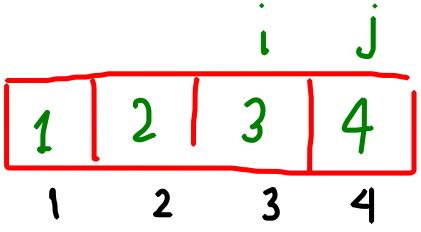
```
1 for (j = 2; j ≤ n; j++) do
2     key = A[j];
3     i = j - 1;
4     while i > 0 and A[i] > key do
5         A[i+1]=A[i];
6         i = i -1;
7     A[i+1] = key;
```



$\Omega(n)$



$O(n^2)$



Key = 2

$C_1 \rightarrow 1$

$C_5 \rightarrow$

$C_6 \rightarrow$

Key = 3

$C_4 \rightarrow 1$

$C_5 \rightarrow$

$C_6 \rightarrow$

Key = 4

$C_4 \rightarrow 1$

$C_5 \rightarrow$

$C_6 \rightarrow$

1 asignación

2. decisión.

3. repetición
   * Iteración → Para / mientras.
   * Recursión

La estategia de diseño de algoritmos denominada **Dividir y conquistar** (o Divide y vencerás) propone resolver problemas de manera recursiva aplicando tres pasos en cada nivel de recursión:

- **Dividir** el problema en un número de subproblemas que son instancias de menor tamaño del mismo problema.
- **Conquistar** los problemas de manera recursiva. Si el tamaño del problema es lo suficientemente pequeño resolverlo de la manera más sencilla.
- **Combinar** las soluciones de los subproblemas en la solución del problema original.

Cuando un algoritmo contiene llamadas recursivas su tiempo de ejecución puede ser descrito gracias a una **recurrencia**.

Definición

Una **relación de recurrencia** *para una secuencia* $F(n)$, $n = 0, 1, 2, ...$ *(también llamada* **recurrencia***) es una ecuación de la forma*

$$F(n) = \phi(n, F(n-1), F(n-2), ..., F(0))$$

donde ϕ *es una fórmula que involucra a* n *y algunas, o todas, las cantidades* $F(n-1), F(n-2), ..., F(0)$.

De manera general, la recurrencia que describe el tiempo de ejecución de un algoritmo que sigue los lineamientos del principio "divide y venceras" es:

Problema más pequeño

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq c \\ aT(\frac{n}{b}) + D(n) + C(n) & \text{de otra forma} \end{cases}$$

Con a la cantidad de subproblemas en los que se divide el problema original, $\frac{1}{b}$ la fracción en la que se reduce el tamaño del problema original, $D(n)$ el tiempo que toma dividir el problema y $C(n)$ el tiempo que toma combinar las soluciones.

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

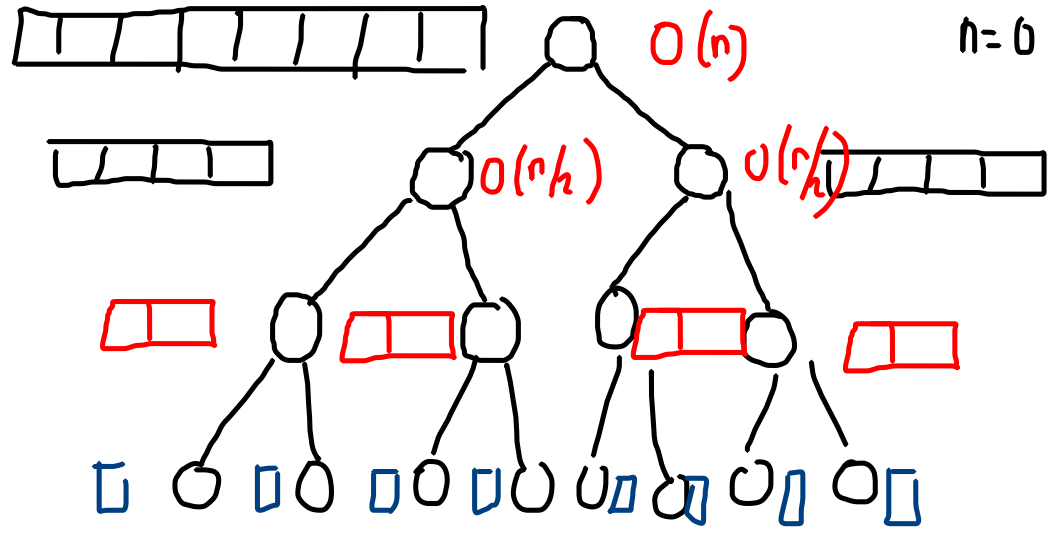
```
MERGE(A, p, q, r)
 1  n1 = q − p + 1
 2  n2 = r − q
 3  let L[1 . . n1 + 1] and R[1 . . n2 + 1] be new arrays
 4  for i = 1 to n1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n2
 7      R[j] = A[q + j]
 8  L[n1 + 1] = ∞
 9  R[n2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(\frac{n}{2}) + \Theta(1) + c(n) & \text{de otra forma} \end{cases}$$

Para resolver la recurrencia existen tres métodos:

1. Método de sustitución (Inducción matemática)
2. Método del árbol de recursión (Método gráfico)
3. Método maestro (exclusivamente para recurrencias de la forma ($T(n) = aT(\frac{n}{b}) + f(n)$).

$2^3 = 8$



$n=0 \rightarrow O(n)$

+

$n=1 \rightarrow O(n)$

+

$n=2 \rightarrow O(n)$

+

$n=3 \rightarrow O(n)$

$2^4 = 16$

$2^5 = 32$

$2^e = n \qquad e = \log_2 n$

número de niveles del árbol del árbol de recurrencia

$(\log_2(n) + 1)$

Costo de cada nivel $O(n)$

$n\log(n) + n$

$O(n\log(n)) + O(n) = O(n\log(n))$

Theorem

Teorema maestro: *Sean* $a \geq 1$ *y* $b > 1$, $f(n)$ *una función y* $T(n) = aT(\frac{n}{b}) + f(n)$ *una recurrencia definida paara los números enteros no negativos con* $\frac{n}{b}$ *entendida como* $\lfloor \frac{n}{b} \rfloor$ *o* $\lceil \frac{n}{b} \rceil$, *se tiene que* $T(n)$ *tiene las siguientes cotas asintóticas:*

1. *Si* $f(n) = O(n^{Log_b(a)-\epsilon})$ *para alguna constante* $\epsilon > 0$, *entonces* $T(n) = O(n^{Log_b(a)})$.
2. *Si* $f(n) = \Theta(n^{Log_b(a)})$, *entonces* $T(n) = O(n^{Log_b(a)} Log(n))$.
3. *Si* $f(n) = \Omega(n^{Log_b(a)+\epsilon})$ *para alguna constante* $\epsilon > 0$ *y, si* $af(\frac{n}{b}) \leq cf(n)$ *para alguna constante* $c < 1$ *y todas las* n *lo suficientemente grandes, entonces* $T(n) = \Theta(f(n))$.

Para determinar el espacio demandado por un algoritmo se suman las demandas de espacio de cuatro componentes

- Espacio del código fuente.
- Espacio de las variables:
  - Espacio de entrada/salida.
  - Espacio auxiliar.
- Espacio de la pila de llamadas (Stack Space).

float → float, double

enteros → short, byte, long, int

Algorithm 4: Multiplicación de matrices.

Input: Arreglo bidimensional A de $n \times t$ números.
Arreglo bidimensional B de $t \times m$ números.

n x t + t x m

Output: Arreglo bidimensional C de $n \times m$ números. → n m

```
1 for i = 1 to n do
2     for j = 1 to m do
3         suma = 0
4         for k = 1 to t do
5             suma = suma + (A[i][k]*B[k][j])
6         C[i][j]=suma
```

i

suma

Θ(1)

Θ(1)

$nt + tm + nm + O(1) + O(1) = O(nt + tm)$

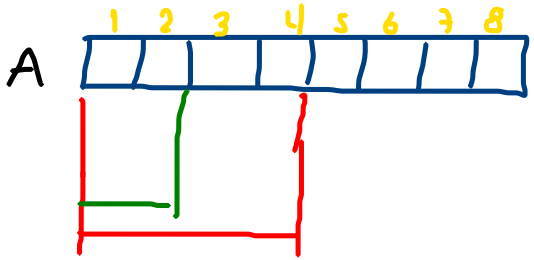
entrada/salida — auxiliares

Algorithm 5: Ordenamiento por Inserción, *Insertion-Sort*

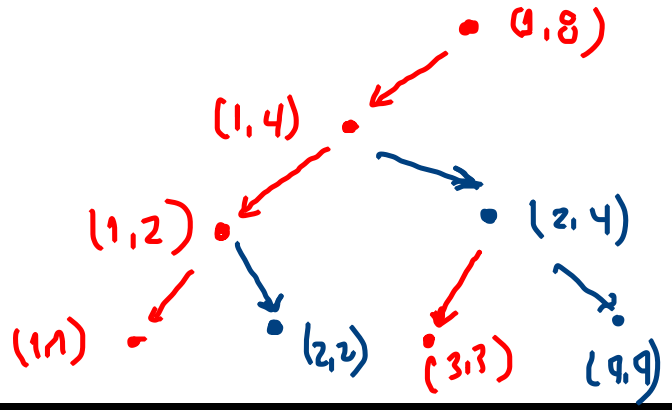
Input: Arreglo unidimensional A de n números.

Output: Arreglo de entrada ordenado.

```
1 for (j = 2; j ≤ n; j++) do
2     key = A[j];
3     i = j - 1;
4     while i > 0 and A[i] > key do
5         A[i+1]=A[i];
6         i = i -1;
7     A[i+1] = key;
```




```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```




```
MERGE(A, p, q, r)
1   n1 = q − p + 1
2   n2 = r − q
3   let L[1 . . n1 + 1] and R[1 . . n2 + 1] be new arrays
4   for i = 1 to n1
5       L[i] = A[p + i − 1]
6   for j = 1 to n2
7       R[j] = A[q + j]
8   L[n1 + 1] = ∞
9   R[n2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```